YubiKit .NET v2

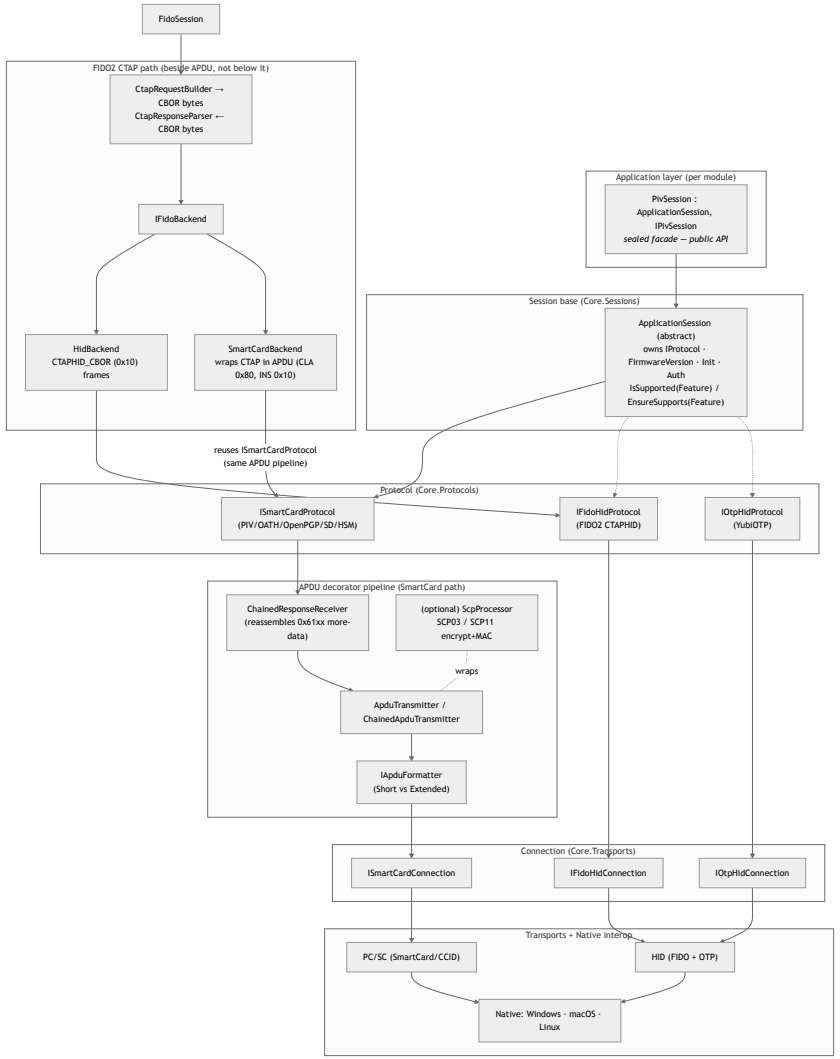
A tour for people who live in the other SDKs

`yubikey-manager` · `yubikit-android` · `yubikit-swift` · `python-fido2`

- **Async on the golden path** — every applet operation is `async`; the few sync members that remain are deliberate lifecycle and transaction escapes
- **Install only what you use** — ten packages, not one monolith; a PIV app ships 742 KiB against v1's unavoidable 890 KiB
- **Native AOT** — ships as a standalone native binary: no JIT, no runtime install, 11.7 MiB resident with the whole SDK linked
- **Event-driven discovery** — OS notifications
- **One session shape** — learn one applet, you know the rest

Branch `yubikit` @ `d04d59aa` · 2026-09-07

Host App → SDK → Session → YubiKey



The pipeline in code

```
// 1. HOST APP asks the SDK for devices
var keys = await YubiKeyManager.FindAllAsync();
IYubiKey key = keys[0];

// 2. SESSION — convenience: the session owns a hidden connection
await using var piv = await key.CreatePivSessionAsync();

// 3. YUBIKEY executes; the session speaks the applet protocol
var cert = await piv.GetCertificateAsync(PivSlot.Authentication);
```

Every layer boundary is `async`. There is no synchronous escape hatch on the golden path.

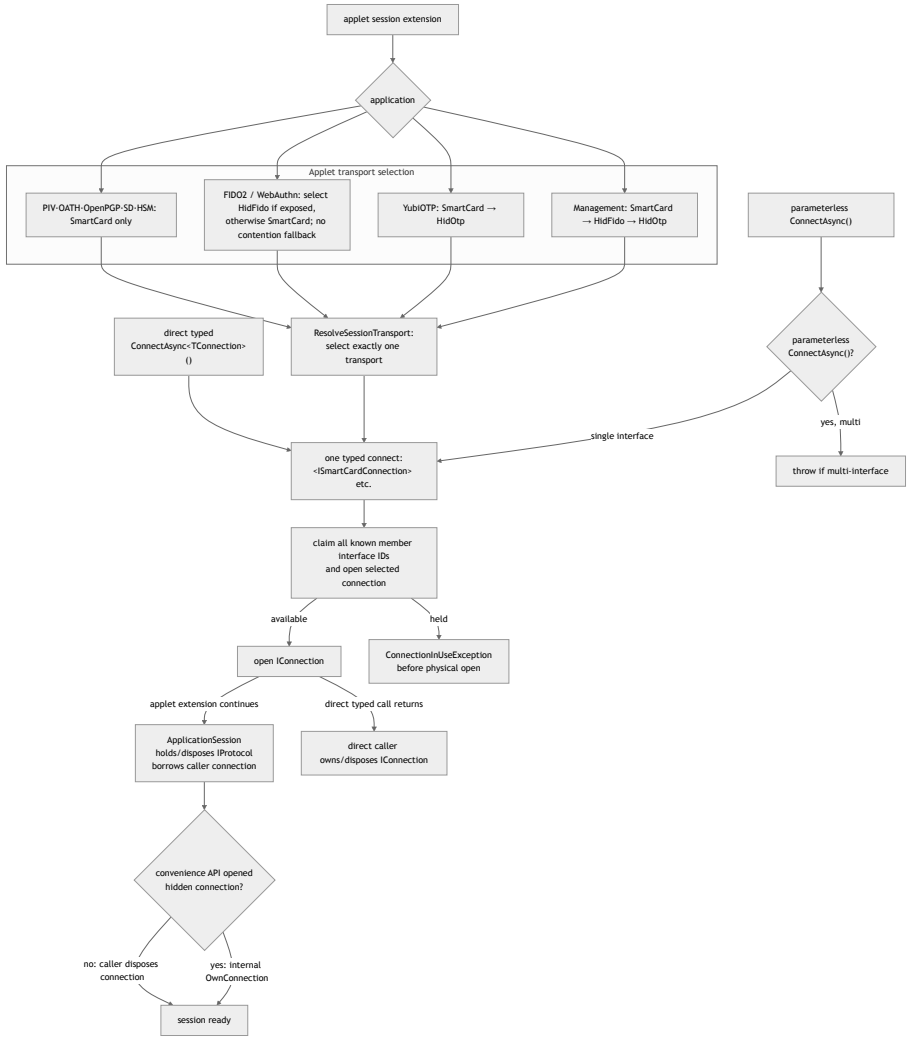
Three ways to open a session

```
// A. Convenience — SDK resolves the transport, owns and disposes the connection
await using var piv = await key.CreatePivSessionAsync();

// B. Convenience + options — force a transport, add a secure channel
await using var piv = await key.CreatePivSessionAsync(
    new SessionCreationOptions
    {
        PreferredConnectionType = ConnectionType.SmartCard,
        ScpKeyParameters = scpKeyParams,
    });

// C. Bring your own connection — you own it, you dispose both
await using var conn = await key.ConnectAsync<ISmartCardConnection>();
await using var piv  = await PivSession.CreateAsync(conn);
await using var mgmt = await ManagementSession.CreateAsync(conn);   // after piv is disposed
```

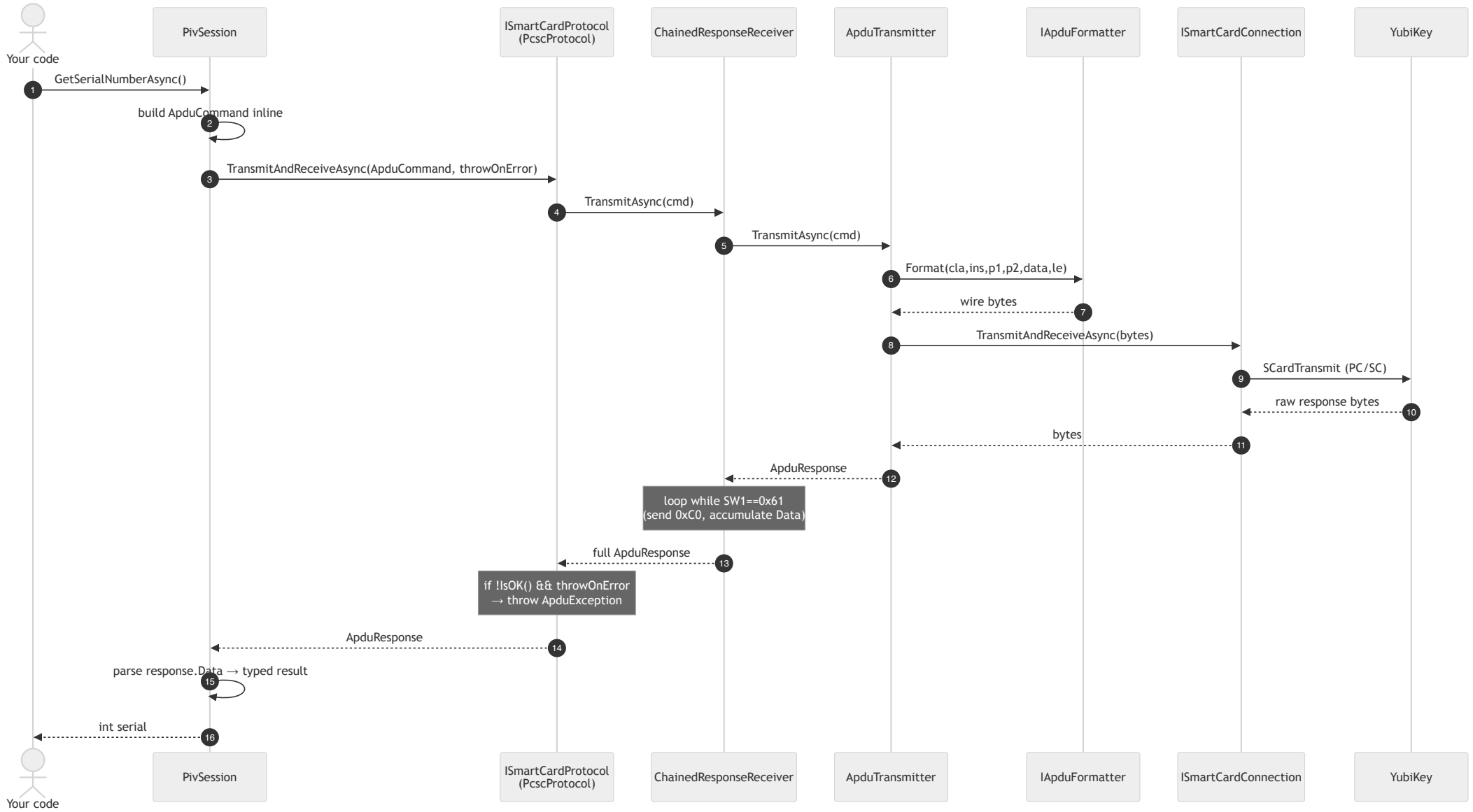
A for one-shot work. **B** when the default transport is wrong, or you need SCP.

C when several applets share one connection — sequentially; one connection admits one live session.

Transports and interfaces

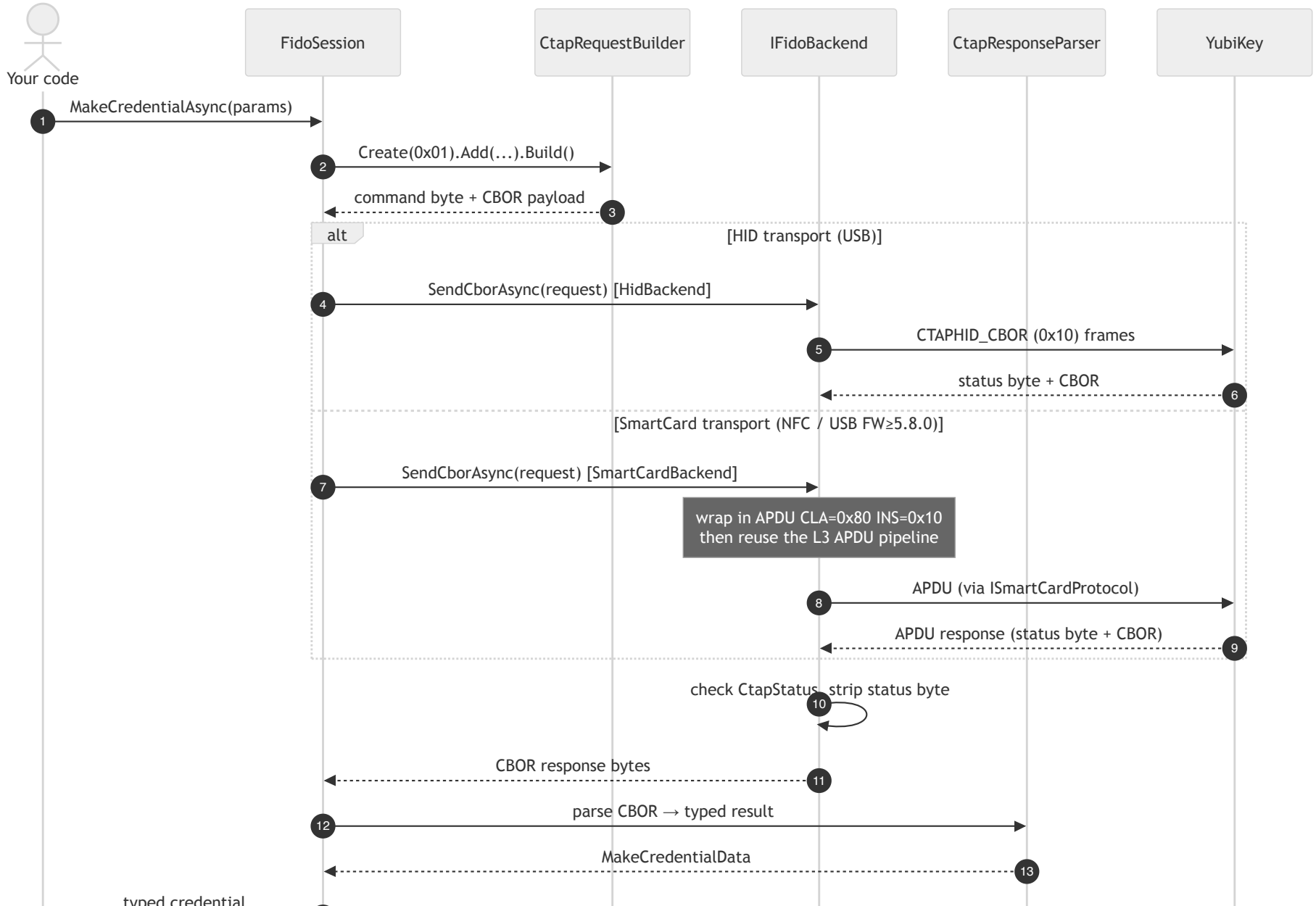


Command invocation: smart card / APDU



PIV · OATH · OpenPGP · Management · SecurityDomain · YubiHSM Auth

Command invocation: FIDO HID / CTAP



Applets are not one-transport-each

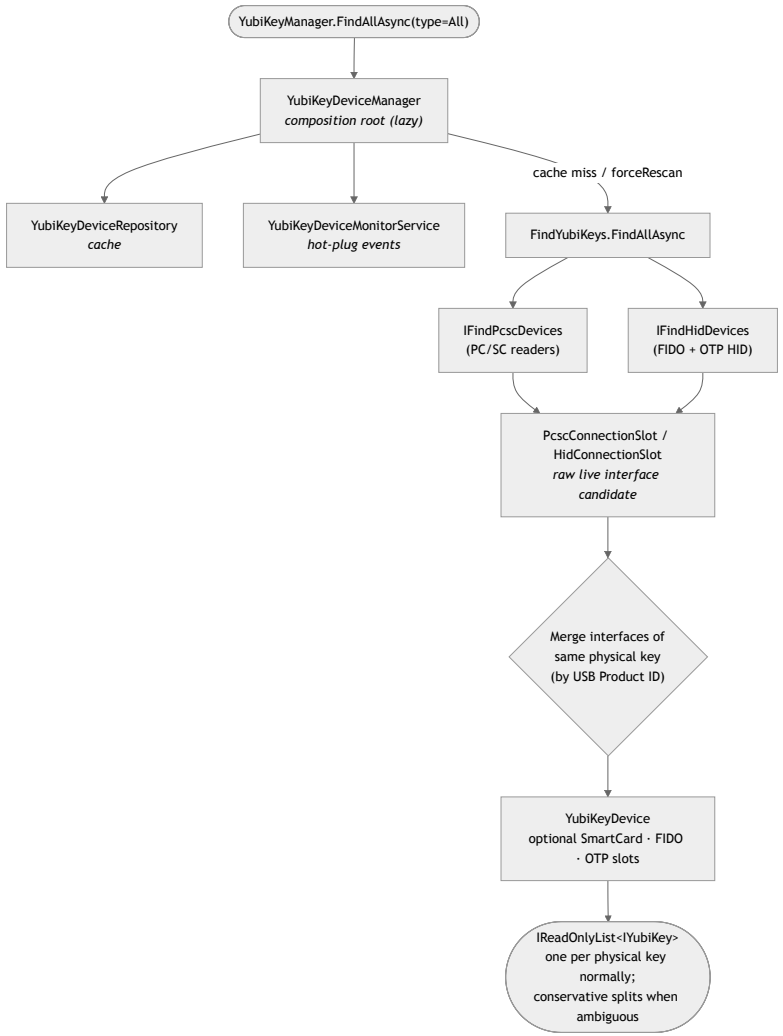
`[Flags] ConnectionType` : `Unknown=0` · `Hid=1` · `HidFido=2` · `HidOtp=4` · `SmartCard=8` · `All`

Applet	Default transport order
Management	`SmartCard` → `HidFido` → `HidOtp`
FIDO2	`HidFido` → `SmartCard`
WebAuthn	inherits FIDO2's order
YubiOTP	`SmartCard` → `HidOtp`
PIV · OATH · OpenPGP · SecurityDomain · YubiHSM	`SmartCard` only

- FIDO2's SmartCard path is **NFC, or USB-CCID on firmware 5.8.0+**.
- **YubiOTP prefers SmartCard**, not HID — on a CCID-enabled key the YubiOTP snippet later in this deck runs over APDU.
- Supplying `ScpKeyParameters` **forces SmartCard** when no preference is given.

```
await using var mgmt = await key.CreateManagementSessionAsync(
    new SessionCreationOptions { PreferredConnectionType = ConnectionType.HidFido });
```

Device discovery



Two operation models

A — One-shot. Ask, act, exit. No monitoring.

```
var keys = await YubiKeyManager.FindAllAsync();
await using var piv = await keys[0].CreatePivSessionAsync();
// ... do the work, then exit
```

B — Long-lived. Start monitoring, react to arrivals for the process lifetime.

```
YubiKeyManager.StartMonitoring();

await foreach (var e in YubiKeyManager.WatchAsync(ct))
{
    if (e.Action == DeviceAction.Added)
        await OnKeyInserted(e.Device);
}
```

Same `IYubiKey`, same sessions. The models differ only in **who keeps the cache fresh.**

Which model serves whom

	A — One-shot	B — Monitored
Who	CLI tools, scripts, CI, installers	Desktop apps, daemons, services, tray UIs
Lifetime	seconds	hours
Cache kept fresh by	**you**, via `forceRescan`	the monitor
`forceRescan: true`	needed on every re-poll	**redundant — drop it**
Cost when idle	none	one listener, no polling

```
// A — polling loop without monitoring: MUST force, or you re-read a stale cache
while (!ct.IsCancellationRequested)
{
    var keys = await YubiKeyManager.FindAllAsync(forceRescan: true);
    ...
    await Task.Delay(1000, ct);
}
```

> **The trap.** Model A without `forceRescan` scans once, then returns that first snapshot **forever**. It looks like it works, because the first call is correct. Either pass `forceRescan: true`, or switch to model B.

`ykman`'s CLI is model A. `yubikit-android` and `yubikit-swift` apps are model B. Most .NET desktop consumers want B; most .NET tooling wants A.

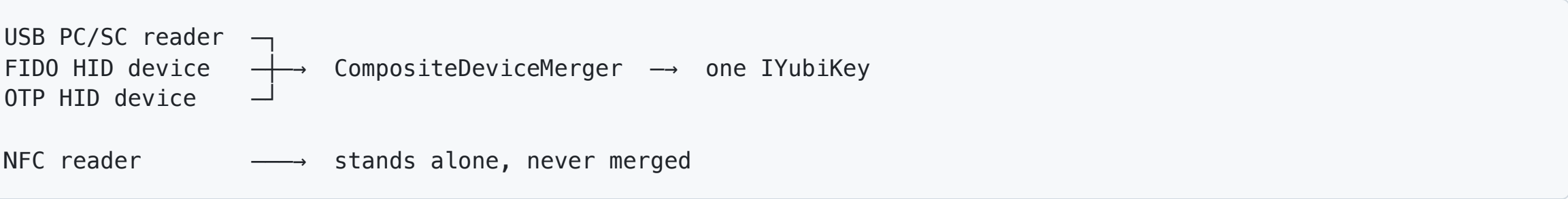
Discovery guarantees

Discovery is **publish-first and degraded-state tolerant** — a key is published as soon as it is enumerated, even if its metadata read has not succeeded. Canonical Rust instead withholds publication until metadata is read, which is why `SerialNumber` can arrive late here and not there.

Both models inherit this. Neither model can promise a key that arrives *during* a scan appears in that scan's result.

Merging: one physical key, many interfaces

A single YubiKey appears as **several OS-level devices** — a PC/SC reader, a FIDO HID device, an OTP HID device. The SDK merges them into one `IYubiKey`.



Merge inputs per interface: `Connection` · `IsUsb` · `Pid` · `Serial` · `DeviceInfo` · `TopologyKey` (Windows Container ID; `null` on macOS and Linux)

What happens to an NFC key

NFC is **discovered and monitored exactly like USB** — the same `SCardGetStatusChange` listener covers NFC readers, so tap and remove raise the same `Added` / `Removed` events, and the same repository cache holds them.

What differs is **grouping**, and only grouping:

- Merging needs USB evidence — Product ID from the reader name, or a Windows Container ID. An NFC reader has neither.
- So an NFC-presented key is **published standalone**, with a transport-shaped `DeviceId` (`pcsc:*`), not a `ykphysical:*` one.
- Tap the same key over USB and NFC at once and you get **two `IYubiKey` objects**. The SDK will not claim they are one physical key, because nothing proved it.
- The honest correlator is `SerialNumber` — read it from both and compare.

This is conservative on purpose: a wrong merge is worse than no merge.

Identity: what is stable, what is not

Surface	Guarantee
`DeviceId`	**Diagnostic only.** Not durable identity.
`SerialNumber`	`null` → value, **never** back to `null`.
Interface-set key	Internal, machine-local, never public
`Equals` / `GetHashCode`	**Referential** — same object, or not equal

- `SerialNumber` may stay `null` **forever**: reads fail, budgets exhaust, and the Security Key series reports no serial at all.
- It can flip `null` → value **after** publication, with **no** device event.
- A key whose interface set changes is **republished as a new object** that inherits nothing from its predecessor.

Who actually wants `DeviceId`?

Its value is that the **prefix names the evidence tier** that produced it:

Shape	Means
`hid:{reader}:{usage}`	a lone HID interface — nothing proved a physical key
`pcsc:*`	a lone smart-card reader, e.g. anything over NFC

Observability: the whole device-event API

```
YubiKeyManager.StartMonitoring();                // events don't flow until this

await foreach (var e in YubiKeyManager.WatchAsync())
{
    Console.WriteLine($"{e.Action}: {e.Device.DeviceId}");
}
```

```
public enum DeviceAction { Added, Removed }
public class DeviceEvent { IYubiKey Device; DeviceAction Action; DateTime Timestamp; }
```

The monitoring **control** surface is five members: `StartMonitoring()`, `StartMonitoring(TimeSpan)`, `StopMonitoring()`, `WatchAsync(ct)`, `IsMonitoring`. `Shutdown()` / `ShutdownAsync()` also stop monitoring as part of tearing the manager down.

How the peers surface device changes

SDK	Attach / detach
.NET v2	`await foreach WatchAsync()` — OS-notification driven
`yubikey-manager` (Python)	**Poll only** — `scan_devices()` in a `sleep` loop
`yubikey-manager` (rust)	`monitor_yubikeys(cb)` — udev / `WM_DEVICECHANGE` / `IOHIDManager`
`yubikit-android`	Callbacks — `Callback<T>.invoke` attach, `setOnClosed` detach
`yubikit-swift`	**Connect-on-demand** — `makeConnection()` polls; `waitUntilClosed()`
`python-fido2`	**None** — `list_devices()` enumerates on call

Two things worth noting: rust has a third variant, `YubiKeyEvent::Changed`, that .NET does not. And Swift's `makeConnection()` is a 1-second poll loop internally, not an OS notification.

Observability: logging

Off by default. No output, no logger, no overhead until you opt in.

```
// Silent — the default
var keys = await YubiKeyManager.FindAllAsync();

// One line, before you touch the SDK
YubiKitLogging.Configure(loggerFactory);
```

Six documented setups: static, DI, ASP.NET Core, `appsettings.json`, Serilog, none. Categories are class names — `Yubico.YubiKit.Piv.PivSession`, `Yubico.YubiKit.Core.Transports.SmartCard.*` — so you can filter per applet or per transport.

Level	What lands there
`Trace`	Raw APDU / CBOR bytes, protocol steps
`Debug`	State transitions, cache updates
`Information`	Session creation, major operations
`Warning` / `Error`	Recoverable fallback / operation failure

Never logged: PINs, PUKs, passwords, private keys, session keys.
Serial numbers and credential IDs are public identifiers and may appear at `Debug`.

Logging: we already agree across the SDKs

SDK	Default	Facade	Enable	Logger obtained
.NET v2	silent	`Microsoft.Extensions.Logging`	`YubiKitLogging.Configure(f)`	**static**
ykman (py)	silent	stdlib `logging`	`init_logging(level)`	per-module
ykman (rust)	silent	`log` crate	install a `log` impl	macros, per call site
yubikit-android	silent	**slf4j**	add a binding	per-class
yubikit-swift	silent	`OSLog`	on by platform	static, per domain

Three conventions we already share, without ever having agreed them:

1. **A facade, never a concrete logger.** Every SDK binds to an abstraction and lets the host pick the sink.
2. **Silent until configured.** No SDK here prints anything out of the box.
3. **Raw protocol bytes go to `Trace`.** Android says so explicitly; .NET puts APDU and CBOR there too.

> **Nobody injects a logger into a session.** All five obtain one statically or per-module. .NET v2's "never inject `ILogger`" is not a .NET quirk — it is the house style everywhere, and it is what keeps the SDK usable without a DI container.
>
> Worth knowing: Android *used* to have a settable static `Logger.setLogger()` and

Going below the applet API: three tiers

Tier	What it gives you	What you take on
0 — Applet session	applet semantics, feature gates, protocol state	nothing
1 — Raw session	APDU/CTAP framing, chaining, overlap refusal, SCP	payload semantics
2 — Raw connection	a pipe	*everything*

```
await using var piv  = await key.CreatePivSessionAsync();           // Tier 0
await using var raw  = await key.CreateRawSmartCardSessionAsync();  // Tier 1
await using var conn = await key.ConnectAsync<ISmartCardConnection>();
var rsp = await conn.TransmitAndReceiveAsync(rawBytes);             // Tier 2
```

Tier 1 does **not** select an applet or apply feature gates. At Tier 2 you own APDU formatting, chaining, response correlation, CRC validation, keep-alive, sequencing, and recovery from partial or cancelled exchanges.

Dropping a tier does **not** drop transport safety — raw sessions still obey one-connection-per-key and one-session-per-connection.

> This is the supported answer for teammates hand-rolling APDUs in `ykman` today: there is a place to do that, and it is Tier 1, not Tier 2.

One session shape — for eight of the nine

```
await using var s = await key.CreateXSessionAsync(options, cancellationToken);
```

- A sealed `XSession` plus a matching `IXSession` interface
- `IYubiKey.CreateXSessionAsync(...)` — session owns a hidden connection
- `XSession.CreateAsync(connection, ...)` — session borrows yours
- Both take `SessionCreationOptions?` then a defaulted `CancellationToken`
- Always `await using`

> **WebAuthn is the deliberate exception.** It is not an applet session. It returns a `WebAuthnClient`, takes a required origin and public-suffix checker, and has its own factory test. The test file says so in as many words:
> *"WebAuthn is not an applet session and has its own factory test."*

Ownership: who disposes the connection

```
// Convenience — the session owns a hidden connection and disposes it
await using var piv = await key.CreatePivSessionAsync();
```

```
// Direct factory — you own the connection, you dispose both
await using var conn = await key.ConnectAsync<ISmartCardConnection>();
await using var mgmt = await ManagementSession.CreateAsync(conn);
var info = await mgmt.GetDeviceInfoAsync();
```

One key admits one live connection. One connection admits one live session — a second `Attach` throws `ConnectionInUseException`. Dispose session N before creating N+1 over the same connection. Prefer `DisposeAsync`; sync `Dispose` blocks to drain.

SDK	Who owns the connection
.NET v2	Both, named explicitly at the call site
ykman (Python)	Caller — `with dev.open_connection(...)`
ykman (rust)	Session **consumes** it; returns `(Error, Connection)` on failure
yubikit-android	Session borrows, but `close()` closes the connection too
yubikit-swift	Caller owns; session never closes it

Management

Read device info, toggle capabilities, write device config.

```
var info = await key.GetDeviceInfoAsync();   // no session needed
```

ykman (Python)

```
mgmt = ManagementSession(conn)
info = mgmt.read_device_info()
```

yubikit-swift

```
let s = try await Management.Session.makeSession(connection: connection)
let info = try await s.getDeviceInfo()
```

Delta: .NET gives Management a device-level shortcut — no session boilerplate for the most common read. It is also the widest applet: three transports, `SmartCard → HidFido → HidOtp`. Swift covers two, with `SmartCardConnection` and `FIDOConnection` overloads.

PIV

Certificates, key pairs, PIN/PUK, attestation.

```
await using var piv = await key.CreatePivSessionAsync();
var cert = await piv.GetCertificateAsync(PivSlot.Authentication);
```

ykman (Python)

```
piv = PivSession(conn)
cert = piv.get_certificate(SLOT.AUTHENTICATION)
```

yubikit-android

```
PivSession piv = new PivSession(smartCardConnection);
X509Certificate cert = piv.getCertificate(Slot.AUTHENTICATION);
```

Delta: all three return a real certificate type — `X509Certificate2`, `x509.Certificate`, `java.security.cert.X509Certificate`. The .NET difference is **nullability**: an empty slot returns `null` rather than throwing.

OATH

TOTP and HOTP credentials, calculate codes, applet password.

```
await using var oath = await key.CreateOathSessionAsync();
var codes = await oath.CalculateAllAsync();
```

ykman (Python)

```
oath = OathSession(conn)
creds = oath.list_credentials()
```

yubikit-android

```
OathSession oath = new OathSession(conn);
Map<Credential, @Nullable Code> codes = oath.calculateCodes();
```

Delta: naming, the closest convergence in the deck — .NET's `IReadOnlyDictionary<Credential, Code?>` and Android's `Map<Credential, @Nullable Code>` express the same touch-required semantics.

OpenPGP

Card data, key slots, sign / decrypt / authenticate, PIN management.

```
await using var pgp = await key.CreateOpenPgpSessionAsync();
var data = await pgp.GetApplicationRelatedDataAsync();
```

ykman (Python)

```
pgp = OpenPgpSession(conn)
data = pgp.get_application_related_data()
```

yubikit-android

```
OpenPgpSession pgp = new OpenPgpSession(conn);
ApplicationRelatedData d = pgp.getApplicationRelatedData();
```

Delta: `yubikit-swift` has **no OpenPGP session at all** — only a `Capability.openPGP` bit. If you work in Swift, this applet is entirely new to you.

FIDO2 — the CTAP2 layer

Raw CTAP2: `GetInfo`, `MakeCredential`, `GetAssertion`, credential management, bio.

```
await using var fido = await key.CreateFidoSessionAsync();
var info = await fido.GetInfoAsync();
```

python-fido2

```
ctap2 = Ctap2(dev)
info = ctap2.info      # cached at construction
```

yubikit-swift

```
let s = try await CTAP2.Session.makeSession(connection: connection)
let info = try await s.getInfo()
```

Delta: `python-fido2` runs `GET_INFO` **inside the constructor** and caches it; .NET keeps construction cheap and the round-trip explicit and cancellable. Note FIDO2 is dual-transport: `HidFido` first, then SmartCard — NFC, or USB-CCID on **firmware 5.8.0+**.

WebAuthn — the layer above CTAP2

Origin checks, client data, attestation, extensions, PIN/UV orchestration.

```
_ = WebAuthnOrigin.TryParse("https://example.com", out var origin);
await using var client = await key.CreateWebAuthnClientAsync(
    origin!, isPublicSuffix: d => d is "com" or "org" or "net");
var reg = await client.MakeCredentialAsync(options, pinBytes);
```

python-fido2

```
result = client.make_credential(create_options["publicKey"])
```

yubikit-swift *(release/1.4.0)*

```
let client = WebAuthn.Client(session: session, origin: try .init("https://example.com"),
                             isPublicSuffix: { publicSuffixList.contains($0) })
let r = try await client.makeCredential(options, authorization: .pin("1234")).value
```

Delta: both .NET and Swift demand an **origin and a public-suffix checker up front** — you cannot accidentally skip origin validation. `python-fido2` also ships the relying-party half (`Fido2Server`); v2 is client-side only.

SecurityDomain (SCP)

GlobalPlatform key management — SCP03 and SCP11, certificates, CA identifiers.

```
await using var sd = await key.CreateSecurityDomainSessionAsync();
var keyInfo = await sd.GetKeyInfoAsync();
```

ykman (Python)

```
sd = SecurityDomainSession(conn)
keys = sd.get_key_information()
```

yubikit-swift

```
let s = try await SecurityDomainSession.makeSession(connection: connection)
let keyInfo = try await s.getKeyInformation()
```

Delta: same operation, three spellings — .NET abbreviates to `GetKeyInfoAsync` where Python and Swift both write *Information*. Bigger point: in .NET, SCP is *also* a creation option on every other applet — pass `ScpKeyParameters` in `SessionCreationOptions` and any session runs over a secure channel.

YubiHSM Auth

Store and use credentials that authenticate to a YubiHSM 2.

```
await using var hsm = await key.CreateHsmAuthSessionAsync();
var creds = await hsm.ListCredentialsAsync();
```

ykman (Python)

```
hsm = HsmAuthSession(conn)
creds = hsm.list_credentials()
```

ykman (rust)

```
let mut s = HsmAuthSession::new(conn).map_err(|(e, _)| e)?;
let creds = s.list_credentials()?;
```

Delta: only .NET, Python and rust implement this. `yubikit-android` has **no** YubiHSM Auth module; `yubikit-swift` has only the `Capability.hsmAuth` bit.

YubiOTP

The two programmable slots — Yubico OTP, static password, HMAC-SHA1 challenge-response.

```
await using var otp = await key.CreateYubiOtpSessionAsync();
var response = await otp.CalculateHmacSha1Async(Slot.Two, challenge);
```

ykman (Python)

```
otp = YubiOtpSession(conn)
r = otp.calculate_hmac_sha1(SLOT.TWO, challenge)
```

yubikit-android

```
YubiOtpSession otp = new YubiOtpSession(otpConnection);
byte[] r = otp.calculateHmacSha1(Slot.TWO, challenge, null);
```

Delta: the one applet where all four SDKs agree almost exactly — same operation, same slot enum, same argument order. The .NET difference is that its session is **dual-transport and prefers SmartCard** (`SmartCard → HidOtp`), so on a CCID-enabled key this runs over APDU, not HID. `yubikit-swift` has no YubiOTP session at all.

Applet coverage across the SDKs

Applet	.NET v2	ykman (py)	ykman (rust)	android	swift
Management	✅	✅	✅	✅	✅
PIV	✅	✅	✅	✅	✅
OATH	✅	✅	✅	✅	✅
OpenPGP	✅	✅	✅	✅	❌
FIDO2 / CTAP2	✅	➡️	✅	✅	✅
WebAuthn	✅	➡️	✅	✅	✅
SecurityDomain	✅	✅	✅	✅	✅
YubiHSM Auth	✅	✅	✅	❌	❌
YubiOTP	✅	✅	✅	✅	❌

➡️ = not implemented in-repo; delegated to `python-fido2`, a hard dependency (`fido2 >=2.0,<3`). That library supplies both `Ctap2` and `Fido2Client` — and, uniquely, `Fido2Server`, the relying-party half no other SDK here ships.

.NET v2 and the rust experiment are the only two with all nine in one repo.

Footprint: monolith vs modular

v1 ships two assemblies. You take all of it, always.

v1 assembly	KiB
`Yubico.YubiKey` — every applet	676
`Yubico.Core` — transports, protocols	215
total, unavoidable	**890**

v2 ships ten. You reference what you use.

Package	KiB		Package	KiB
Core	**577**		WebAuthn	130
Fido2	197		OpenPgp	110
Piv	165		Oath	62
SecurityDomain	56		YubiOtp	56
YubiHsm	51		Management	37

Scenario	KiB	vs v1
v2, PIV app (Core + Piv)	**742**	−148, **17 % smaller**

Why is the saving only 17 %?

Fair question. Modularity should win bigger. Two things cap it.

1. `Core` is the floor, and it grew 2.7×. v1's shared layer was 215 KiB; v2's is 577 KiB. A PIV app pays that before it pays for PIV. The nine applets are cheap (37–197 KiB each) — skipping eight of them is where the 148 KiB actually comes from.

2. v2 emits far more IL per line of source.

	v1	v2
Source	113,439 LOC	77,512 LOC
Assemblies	890 KiB	1,441 KiB
Bytes per line	**8.0**	**19.0** — 2.4×
`async` methods	**1**	**488**
`record` types	—	94

v2 has **35 % less source** and **62 % more binary**. Every `async` method compiles to a generated state-machine type — fields for each local that survives an `await`, a `MoveNext` switch, builder plumbing. Every `record` generates `Equals`, `GetHashCode`, `ToString`, `PrintMembers`, `Clone`, `Deconstruct` and two operators.

So the size story is a trade, not a win. async-everywhere and value-semantics cost

Footprint: Native AOT

Verification host linking all ten libraries — a whole-SDK upper bound.

	Native AOT	Framework-dependent
Executable	**3.11 MiB**	—
NativeShims sidecar	3.71 MiB	3.71 MiB
Peak RSS	**11.7 MiB**	51.5 MiB → **4.4× less**
CPU (user + sys)	**~20 ms**	~170 ms → **~an order of magnitude less**
Wall, median of 10	528 ms	628 ms

What "11.7 MiB resident" means. Native AOT compiles to a standalone native executable — no JIT, no runtime install, no warm-up. The process starts already compiled, so it allocates a fraction of the managed heap a JIT'd process needs. For a CLI, an installer, a service, or anything short-lived, that is the difference between feeling instant and feeling like a .NET app.

Note this cuts against the IL-size trade above: the IL grows, but AOT never ships it.

Footprint: how these were collected

Machine. Apple M1, 8 cores, 16 GB, macOS 15.7.7, .NET SDK 10.0.100, RID `osx-arm64`.
v2 @ `d04d59aa`. v1 @ `fd16960a`, `netstandard2.1`. **6 YubiKeys physically attached.**

Number	Provenance
v2 assembly sizes	**Measured** — `stat` on `bin/Release/net10.0/*.dll`
v1 assembly sizes	**Measured** — `dotnet build -c Release -f netstandard2.1` at `fd16960a`
LOC, async and record counts	**Measured** — `find` + `wc` and `rg` over `src`, excluding `obj/` and `bin/`
AOT exe, RSS, CPU, wall	**Measured** — `dotnet publish -r osx-arm64 -p:PublishAot=true` of `verification/NativeAotVerification`; `/usr/bin/time -l`; wall = median of 10 after 3 warm-ups
Framework-dependent baseline	**Measured** — same project, `-p:PublishAot=false --self-contained false`
AOT link coverage, support contract	**Quoted** — PR #578, `docs/NATIVE-AOT.md`
Recurring AOT CI	**CI** — `native-aot.yml` run `34121554932` on `yubikit`, macOS arm64, **hardware-free**

⚠️
Caveats. Wall time is dominated by I/O enumerating six attached keys, not startup: subtracting CPU leaves 508 ms (AOT) and 458 ms (framework-dependent). `/usr/bin/time -l` quantises CPU to 10 ms, so 20 vs 170 ms is two ticks against

Takeaways

One shape, eight applet sessions — test-enforced, not conventional.
WebAuthn is the deliberate exception: it returns a client, not a session.

Ownership is explicit. Convenience owns the connection, the direct factory borrows it.
One connection, one live session, enforced at runtime.

Device events are one API. `StartMonitoring()` + `await foreach WatchAsync()`.

Identity is honest about what it cannot promise. `DeviceId` is diagnostic;
`SerialNumber` may be `null` forever and can arrive late without an event.

AOT is real. 11.7 MiB resident with the whole SDK linked, ~4.4× less than JIT.

`docs/architecture/` · `docs/usage/device-discovery.md` · `docs/NATIVE-AOT.md`
Run the single-file WebAuthn + previewSign demo

Status **2.0.0-alpha.2** — public API still in `PublicAPI.Unshipped.txt`, so breaking
changes are still cheap. Now is the time to complain.
<script src="assets/vendor/panzoom.min.js"></script> <script src="assets/deck-zoom.js"></script>